Formal Verification Report: Bridge X

- Date: March 13th, 2026
- Audit Repo: https://github.com/Cyfrin/audit-2026-02-bridgex
- Client Repo: https://github.com/NerdUnited-NodeGovernance/bridge-x-contracts
- Audit Commit: 406c951d3b3ea02d80e9d725c65005857947cae7
- Mitigation Commit: 8153e1ee1002cb440cf165a73adc3177289e27c4
- Author: Specialist AI FV-Certora v1.0.9 created by Dacian
- Human Auditor: Dacian (Cyfrin private audit)
- Certora Prover version: 8.8.1

Table of Contents

About Bridge X

Bridge X is a bi-directional bridge protocol enabling token transfers between private GoQuorum chains and public EVM chains. The protocol consists of three core contracts:

- `Token` — A custom ERC-20 token with owner-controlled issuance, two-step ownership transfer, max supply cap, and no burn functionality. Used on public chains to wrap private chain gas tokens as ERC-20.
- `PrivateChainBridge` — Deployed on the private GoQuorum chain. Users lock native gas tokens via `lockTokens`, and multi-sig releasers sign cross-chain releases via `signRelease`. Uses a native ETH vault for token custody.
- `PublicBridge` — Deployed on public EVM chains (Ethereum, BSC, Base). Users lock ERC-20 tokens via `lockTokens`, and releasers sign releases that mint or transfer tokens from a vault. Includes a try/catch around `Token::mint` to handle maxSupply being reached.

Both bridge contracts share a common architecture: owner-managed releaser sets, multi-signature release authorization, two-step ownership transfer, pause functionality, configurable bridge/release fees, and a pending release queue for fee-gated withdrawals.

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

Invariants — System-wide properties that MUST always hold true. These are parametric — automatically verified against every external function in the contract. Once proven, invariants serve as trusted assumptions via `requireInvariant`.

Parametric Rules — Rules verified against every non-view external function using `method f` with `calldataarg args`. Used for properties like "only mint can increase totalSupply" or "counter values never decrease."

Access Control Rules — Rules verifying that state-changing functions revert when the caller lacks the required role. Uses the `@withrevert` pattern: call the function, then `assert !lastReverted => hasRole(...)`.

Revert Condition Rules — Rules verifying that functions revert under specific invalid conditions (zero inputs, paused state, duplicate signatures, etc.). Uses `@withrevert` followed by `assert lastReverted`.

Integrity Rules — Rules verifying that successful function calls produce the correct state changes (e.g., transfer moves exact amounts, signature count increments by exactly 1, ownership transfer sets correct values).

Conservation Rules — Rules verifying that value is neither created nor destroyed during operations (e.g., ETH balance after `payReleaseFee` is bounded by the amount received via `msg.value`).

Sanity (Satisfy) Rules — Lightweight reachability checks ensuring functions are not vacuously verified. Uses `satisfy true` to confirm at least one non-reverting execution path exists. Also used for finding validation — proving that reported bugs are reachable.

Key Modeling Decisions

- **External calls summarized as NONDET**: Low-level `.call{value}` (in `payReleaseFee`) and `.transfer` (in `lockTokens`) are resolved as `NONDET` via `unresolved external in _._ => NONDET`. This preserves native ETH balance semantics while preventing storage havoc that would create false counterexamples.
- **Token/Vault external calls summarized**: On `PublicBridge`, ERC-20 methods (`transfer`, `transferFrom`, `balanceOf`, `mint`) and vault `release` are summarized as `NONDET` since the prover verifies bridge logic, not token/vault internals. On `PrivateChainBridge`, only vault `release` is summarized.
- **Loop iteration bound**: Both bridge specs use `loop_iter: 3` and `optimistic_loop: true` because `removeReleaser` uses a swap-and-pop loop and `payReleaseFee` iterates over the releasers array. Rules involving these functions add explicit `require length <= 3` constraints marked as `UNSAFE`.
- **Private variable tracking via ghost + hooks**: The `_nonce` variable is `private` in both bridge contracts. It is tracked using a ghost variable with both `Sstore` and `Sload` hooks to maintain synchronization between the ghost and actual storage in arbitrary starting states.
- **Optimistic fallback**: Both bridge configs use `optimistic_fallback: true` because the contracts have no `receive`/`fallback` functions, and the prover needs to handle ETH transfers to the contract address.

Project Structure

```
certora/
  conf/
      Token.conf
      PrivateChainBridge.conf
      PublicBridge.conf
  harnesses/
      TokenHarness.sol            # Exposes private: owner, pendingOwner, issuer, maxSupply
      PrivateChainBridgeHarness.sol  # Exposes: releasers.length, pendingReleases, vault address
      PublicBridgeHarness.sol     # Exposes: releasers.length, pendingReleases, vault/token address
  specs/
      Token.spec                  # 16 properties: ERC-20 invariants, access control, ownership
      PrivateChainBridge.spec     # 50 properties: bridge operations, multi-sig, fees, pausing
      PublicBridge.spec           # 49 properties: bridge operations, multi-sig, fees, pausing
  invariants.md
  README.md
  README.pdf
```

Harness Contracts

- `TokenHarness` — Inherits `Token`. Exposes `owner`, `pendingOwner`, `issuer`, and `maxSupply` via getter functions since these are `internal`/`private` in the base contract.
- `PrivateChainBridgeHarness` — Inherits `PrivateChainBridge`. Exposes `releasers.length`, `pendingReleasesByRecipient[]` `pendingReleasesByRecipient[].length`, and `vault` address for CVL rules that need array/struct access.
- `PublicBridgeHarness` — Inherits `PublicBridge`. Same as above plus `token` address getter.

Verification Properties

119 properties across 3 contracts: 2 invariants, 39 parametric rules, 29 access control rules, 20 revert condition rules, 17 integrity rules, 2 conservation rules, 10 sanity rules.

`Token` (16 properties)

Covers ERC-20 token invariants, access control for minting and ownership, transfer conservation, zero-address rejection, two-step ownership integrity, and allowance correctness. Immutability of `maxSupply` and `decimals` is now enforced by the Solidity `immutable` keyword at the compiler level.

ID	Name	Type	Description	Audit	Mitig.
T-1	`totalSupplyIsSumOfBalances`	Invariant	`totalSupply` always equals the sum of all `balanceOf` values (ghost sum tracking)	✓	✓
T-2	`totalSupplyBoundedByMaxSupply`	Invariant	`totalSupply` never exceeds `maxSupply`	✓	✓
T-3	`totalSupplyNeverDecreases`	Parametric	No function can decrease `totalSupply` (no burn exists)	✓	✓
T-4	`onlyIssuerCanMint`	Access Control	`mint` reverts when caller is not `issuer`	✓	✓
T-5	`onlyOwnerCanSetIssuer`	Access Control	`setIssuer` reverts when caller is not `owner`	✓	✓
T-6	`onlyPendingOwnerCanConfirm`	Access Control	`confirmOwnership` reverts when caller is not `pendingOwner`	✓	✓
T-7	`transferConservation`	Integrity	`transfer` decreases sender and increases recipient by exactly `value`	✓	✓
T-8	`transferToZeroReverts`	Revert Condition	`transfer` to `address(0)` always reverts	✓	✓
T-9	`mintToZeroReverts`	Revert Condition	`mint` to `address(0)` always reverts	✓	✓
T-10	`mintReachable`	Sanity	`mint` has at least one non-reverting execution path	✓	✓
T-11	`transferReachable`	Sanity	`transfer` has at least one non-reverting execution path	✓	✓
T-F1	`setIssuerRevertsForZero`	Revert Condition	**Inv 5 / I-7 FIXED**: `setIssuer(0)` now reverts — validates the fix for `setIssuer` previously allowing `address(0)`	✗	✓
T-43	`ownerOnlyChangesViaConfirmOwnership`	Parametric	`owner` only changes via `confirmOwnership`	✓	✓
T-44	`pendingOwnerOnlyChangesViaExpected`	Parametric	`pendingOwner` only changes via `transferOwnership` or `confirmOwnership`	✓	✓
T-54	`confirmOwnershipIntegrity`	Integrity	After `confirmOwnership`, owner equals previous `pendingOwner` and `pendingOwner` is cleared	✓	✓
T-55	`transferFromAllowanceCorrectness`	Integrity	`transferFrom` decreases allowance by exactly the transferred value	✓	✓

`PrivateChainBridge` (50 properties)

Covers releaser management, multi-signature release authorization, signature monotonicity, processed hash irreversibility, access control for all admin functions, pause enforcement, fee invariants, nonce monotonicity, ETH conservation, state variable modification authorization, and ownership transfer integrity.

ID	Name	Type	Description	Audit	Mitig.
B-10	`isReleaserOnlyChangedByAddRemove`	Parametric	`isReleaser` mapping only changes via `addReleaser` or `removeReleaser`	✓	✓
B-12	`setRequiredSignaturesInBounds`	Integrity	After `setRequiredSignatures`, value is `>= 1` and `<= releasers.length`	✓	✓

ID	Name	Type	Description	Audit	Mitig.
B-13	`signatureCountIncrementsOnSign`	Integrity	`signatureCount` increments by exactly 1 when a new signature is recorded	✓	✓
B-14a	`signatureCountMonotonicity`	Parametric	`signatureCount` never decreases for any hash	✓	✓
B-14b	`signaturesNeverFlipBack`	Parametric	Once `signatures[hash][signer]` is true, it never flips back to false	✓	✓
B-15	`processedHashesIrreversible`	Parametric	`processedHashes` never flips from true back to false	✓	✓
B-17	`duplicateSignatureReverts`	Revert Condition	Signing the same release twice with the same signer reverts	✓	✓
B-18	`eligibleBridgeReleasesOnlyChangedByExpected`	Property	`eligibleBridgeReleases` only changes via `payReleaseFee`, `signRelease`, `forceProcessPendingRelease`, or `processPendingReleasesFor`	✓	✓
B-24a	`onlyOwnerCanAddReleaser`	Access Control	`addReleaser` reverts when caller is not owner	✓	✓
B-24b	`onlyOwnerCanRemoveReleaser`	Access Control	`removeReleaser` reverts when caller is not owner	✓	✓
B-24c	`onlyOwnerCanSetRequiredSignatures`	Access Control	`setRequiredSignatures` reverts when caller is not owner	✓	✓
B-24d	`onlyOwnerCanSetReleaseFee`	Access Control	`setReleaseFee` reverts when caller is not owner	✓	✓
B-24e	`onlyOwnerCanSetBridgeFee`	Access Control	`setBridgeFee` reverts when caller is not owner	✓	✓
B-24f	`onlyOwnerCanSetBridgeFeeReceiver`	Access Control	`setBridgeFeeReceiver` reverts when caller is not owner	✓	✓
B-24g	`onlyOwnerCanPause`	Access Control	`pause` reverts when caller is not owner	✓	✓
B-24h	`onlyOwnerCanUnpause`	Access Control	`unpause` reverts when caller is not owner	✓	✓
B-24i	`onlyOwnerCanTransferOwnership`	Access Control	`transferOwnership` reverts when caller is not owner	✓	✓
B-24j	`onlyOwnerCanForceProcess`	Access Control	`forceProcessPendingRelease` reverts when caller is not owner	✓	✓
B-24k	`onlyOwnerCanSetChainId`	Access Control	`setChainId` reverts when caller is not owner	✓	✓
B-25	`onlyReleaserCanSign`	Access Control	`signRelease` reverts when caller is not a releaser	✓	✓
B-26a	`lockTokensRevertsWhenPaused`	Revert Condition	`lockTokens` reverts when bridge is paused	✓	✓
B-26b	`signReleaseRevertsWhenPaused`	Revert Condition	`signRelease` reverts when bridge is paused	✓	✓
B-26c	`payReleaseFeeRevertsWhenPaused`	Revert Condition	`payReleaseFee` reverts when bridge is paused	✓	✓
B-26d	`processPendingReleasesForRevertsWhenPaused`	Revert Condition	`processPendingReleasesFor` reverts when bridge is paused		✓
B-27	`onlyPendingOwnerCanAccept`	Access Control	`acceptOwnership` reverts when caller is not `pendingOwner`	✓	✓
B-28	`releaseFeeAlwaysPositive`	Parametric	`releaseFee` remains `> 0` once set to a positive value	✓	✓
B-29	`bridgeFeeReceiverNeverZero`	Parametric	`bridgeFeeReceiver` never becomes `address(0)` once set to non-zero	✓	✓
B-32	`nonceOnlyIncreases`	Parametric	`_nonce` (tracked via ghost + dual hooks) never decreases	✓	✓

ID	Name	Type	Description	Audit	Mitig.
B-33	`alreadyProcessedReverts`	Revert Condition	`signRelease` reverts for already processed source chain tx hash	✓	✓
B-35a	`lockTokensRevertsForPrivateDestination`	Revert Condition	`lockTokens` reverts when `destinationChain` is Private (cannot bridge to self)	✓	✓
B-35b	`lockTokensRevertsForZeroValue`	Revert Condition	`lockTokens` reverts when `msg.value` is 0	✓	✓
B-M1	`lockTokensRevertsForUnconfiguredChain`	Revert Condition	**Inv 35 / M-1 FIXED**: `lockTokens` now reverts when destination chain is not configured (`chainId == 0`)	✗	✓
B-37	`payReleaseFeeEthConservation`	Conservation	`payReleaseFee` does not create ETH — bridge balance increase bounded by `msg.value`	✓	✓
B-38	`noEthAccumulationInBridge`	Parametric	Bridge native balance does not increase after any function call	✓	✓
B-43	`ownerOnlyChangesViaAcceptOwnership`	Parametric	`owner` only changes via `acceptOwnership`	✓	✓
B-44	`pendingOwnerOnlyChangesViaExpected`	Parametric	`pendingOwner` only changes via `transferOwnership` or `acceptOwnership`	✓	✓
B-45	`pausedOnlyChangesViaPauseUnpause`	Parametric	`paused` only changes via `pause` or `unpause`	✓	✓
B-46	`releaseFeeOnlyChangesViaSetReleaseFee`	Parametric	`releaseFee` only changes via `setReleaseFee`	✓	✓
B-47	`bridgeFeeOnlyChangesViaSetBridgeFee`	Parametric	`bridgeFee` only changes via `setBridgeFee`	✓	✓
B-48	`bridgeFeeReceiverOnlyChangesViaSetBri`	Parametric	`bridgeFeeReceiver` only changes via `setBridgeFeeReceiver`	✓	✓
B-49	`requiredSignaturesOnlyChangesViaSetRe`	Parametric	`requiredSignatures` only changes via `setRequiredSignatures`	✓	✓
B-50	`chainIdsOnlyChangesViaSetChainId`	Parametric	`chainIds` only changes via `setChainId`	✓	✓
B-51	`processedHashesOnlySetBySignRelease`	Parametric	`processedHashes` only changes via `signRelease`	✓	✓
B-52	`addReleaserIntegrity`	Integrity	After `addReleaser`: `isReleaser` is true and array length increases by 1	✓	✓
B-53a	`removeReleaserIntegrity`	Integrity	After `removeReleaser`: `isReleaser` is false	✓	✓
B-53b	`removeReleaserArrayShrinks`	Integrity	After `removeReleaser`: array length does not increase	✓	✓
B-54	`acceptOwnershipIntegrity`	Integrity	After `acceptOwnership`: owner equals previous `pendingOwner`, `pendingOwner` cleared	✓	✓
B-S1	`lockTokensReachable`	Sanity	`lockTokens` has at least one non-reverting execution path	✓	✓
B-S2	`signReleaseReachable`	Sanity	`signRelease` has at least one non-reverting execution path	✓	✓
B-S3	`payReleaseFeeReachable`	Sanity	`payReleaseFee` has at least one non-reverting execution path	✓	✓
B-S4	`processPendingReleasesForReachable`	Sanity	`processPendingReleasesFor` has at least one non-reverting execution path		✓
B-F1	`removeReleaserCanBreakThreshold`	Integrity	**Inv 12 / L-3**: `removeReleaser` can reduce releasers below `requiredSignatures`, making releases unprocessable	✗	✗

`PublicBridge` (49 properties)

Covers the same bridge operation properties as `PrivateChainBridge` plus `PublicBridge`-specific destination chain validation (must be Private). Immutability of `vault` and `token` addresses is now enforced by the Solidity `immutable` keyword at the compiler level.

ID	Name	Type	Description	Audit	Mitig.
P-10	`isReleaserOnlyChangedByAddRemove`	Parametric	`isReleaser` mapping only changes via `addReleaser` or `removeReleaser`	✓	✓
P-12	`setRequiredSignaturesInBounds`	Integrity	After `setRequiredSignatures`, value is `>= 1` and `<= releasers.length`	✓	✓
P-13	`signatureCountIncrementsOnSign`	Integrity	`signatureCount` increments by exactly 1 when a new signature is recorded	✓	✓
P-14a	`signatureCountMonotonicity`	Parametric	`signatureCount` never decreases for any hash	✓	✓
P-14b	`signaturesNeverFlipBack`	Parametric	Once `signatures[hash][signer]` is true, it never flips back to false	✓	✓
P-15	`processedHashesIrreversible`	Parametric	`processedHashes` never flips from true back to false	✓	✓
P-17	`duplicateSignatureReverts`	Revert Condition	Signing the same release twice with the same signer reverts	✓	✓
P-18	`eligibleBridgeReleasesOnlyChangedByExpected`	Parametric	`eligibleBridgeReleases` only changes via `payReleaseFee`, `signRelease`, `forceProcessPendingRelease`, or `processPendingReleasesFor`	✓	✓
P-24a	`onlyOwnerCanAddReleaser`	Access Control	`addReleaser` reverts when caller is not owner	✓	✓
P-24b	`onlyOwnerCanRemoveReleaser`	Access Control	`removeReleaser` reverts when caller is not owner	✓	✓
P-24c	`onlyOwnerCanSetRequiredSignatures`	Access Control	`setRequiredSignatures` reverts when caller is not owner	✓	✓
P-24d	`onlyOwnerCanSetReleaseFee`	Access Control	`setReleaseFee` reverts when caller is not owner	✓	✓
P-24e	`onlyOwnerCanSetBridgeFee`	Access Control	`setBridgeFee` reverts when caller is not owner	✓	✓
P-24f	`onlyOwnerCanSetBridgeFeeReceiver`	Access Control	`setBridgeFeeReceiver` reverts when caller is not owner	✓	✓
P-24g	`onlyOwnerCanPause`	Access Control	`pause` reverts when caller is not owner	✓	✓
P-24h	`onlyOwnerCanUnpause`	Access Control	`unpause` reverts when caller is not owner	✓	✓
P-24i	`onlyOwnerCanTransferOwnership`	Access Control	`transferOwnership` reverts when caller is not owner	✓	✓
P-24j	`onlyOwnerCanForceProcess`	Access Control	`forceProcessPendingRelease` reverts when caller is not owner	✓	✓
P-24k	`onlyOwnerCanSetChainId`	Access Control	`setChainId` reverts when caller is not owner	✓	✓
P-25	`onlyReleaserCanSign`	Access Control	`signRelease` reverts when caller is not a releaser	✓	✓
P-26a	`lockTokensRevertsWhenPaused`	Revert Condition	`lockTokens` reverts when bridge is paused	✓	✓
P-26b	`signReleaseRevertsWhenPaused`	Revert Condition	`signRelease` reverts when bridge is paused	✓	✓
P-26c	`payReleaseFeeRevertsWhenPaused`	Revert Condition	`payReleaseFee` reverts when bridge is paused	✓	✓
P-26d	`processPendingReleasesForRevertsWhenPaused`	Revert Condition	`processPendingReleasesFor` reverts when bridge is paused		✓
P-27	`onlyPendingOwnerCanAccept`	Access Control	`acceptOwnership` reverts when caller is not `pendingOwner`	✓	✓
P-28	`releaseFeeAlwaysPositive`	Parametric	`releaseFee` remains `> 0` once set to a positive value	✓	✓

ID	Name	Type	Description	Audit	Mitig.
P-29	`bridgeFeeReceiverNeverZero`	Parametric	`bridgeFeeReceiver` never becomes `address(0)` once set to non-zero	✓	✓
P-32	`nonceOnlyIncreases`	Parametric	`_nonce` (tracked via ghost + dual hooks) never decreases	✓	✓
P-33	`alreadyProcessedReverts`	Revert Condition	`signRelease` reverts for already processed source chain tx hash	✓	✓
P-34	`lockTokensRequiresPrivateDestination`	Revert Condition	`lockTokens` on `PublicBridge` reverts when `destinationChain != Private`	✓	✓
P-37	`payReleaseFeeEthConservation`	Conservation	`payReleaseFee` does not create ETH — bridge balance increase bounded by `msg.value`	✓	✓
P-38	`noEthAccumulationInBridge`	Parametric	Bridge native balance does not increase after any function call	✓	✓
P-43	`ownerOnlyChangesViaAcceptOwnership`	Parametric	`owner` only changes via `acceptOwnership`	✓	✓
P-44	`pendingOwnerOnlyChangesViaExpected`	Parametric	`pendingOwner` only changes via `transferOwnership` or `acceptOwnership`	✓	✓
P-45	`pausedOnlyChangesViaPauseUnpause`	Parametric	`paused` only changes via `pause` or `unpause`	✓	✓
P-46	`releaseFeeOnlyChangesViaSetReleaseFee`	Parametric	`releaseFee` only changes via `setReleaseFee`	✓	✓
P-47	`bridgeFeeOnlyChangesViaSetBridgeFee`	Parametric	`bridgeFee` only changes via `setBridgeFee`	✓	✓
P-48	`bridgeFeeReceiverOnlyChangesViaSetBri`	Parametric	`bridgeFeeReceiver` only changes via `setBridgeFeeReceiver`	✓	✓
P-49	`requiredSignaturesOnlyChangesViaSetter`	Parametric	`requiredSignatures` only changes via `setRequiredSignatures`	✓	✓
P-50	`chainIdsOnlyChangesViaSetChainId`	Parametric	`chainIds` only changes via `setChainId`	✓	✓
P-51	`processedHashesOnlySetBySignReleas`	Parametric	`processedHashes` only changes via `signRelease`	✓	✓
P-52	`addReleaserIntegrity`	Integrity	After `addReleaser`: `isReleaser` is true and array length increases by 1	✓	✓
P-53a	`removeReleaserIntegrity`	Integrity	After `removeReleaser`: `isReleaser` is false	✓	✓
P-53b	`removeReleaserArrayShrinks`	Integrity	After `removeReleaser`: array length does not increase	✓	✓
P-54	`acceptOwnershipIntegrity`	Integrity	After `acceptOwnership`: owner equals previous `pendingOwner`, `pendingOwner` cleared	✓	✓
P-S1	`lockTokensReachable`	Sanity	`lockTokens` has at least one non-reverting execution path	✓	✓
P-S2	`signReleaseReachable`	Sanity	`signRelease` has at least one non-reverting execution path	✓	✓
P-S3	`payReleaseFeeReachable`	Sanity	`payReleaseFee` has at least one non-reverting execution path	✓	✓
P-S4	`processPendingReleasesForReachable`	Sanity	`processPendingReleasesFor` has at least one non-reverting execution path		✓
P-F1	`removeReleaserCanBreakThreshold`	Integrity	**Inv 12 / L-3**: `removeReleaser` can reduce releasers below `requiredSignatures`, making releases unprocessable	✗	✗
P-F3	`lockTokensRevertsForZeroAmount`	Revert Condition	**Inv 34 / I-8 FIXED**: `lockTokens(0)` now reverts — validates the fix for zero-amount bridge operations	✗	✓

Assumptions - Safe

These constraints reflect real-world conditions and do not exclude valid attack scenarios:

- **Environment constraints**: `e.msg.sender != currentContract` — contracts cannot call their own external functions. `e.msg.sender != 0` — EVM invariant that `msg.sender` is never `address(0)`.
- **Ghost-storage synchronization**: `Sload` hooks on `_nonce` use `require g_nonce == val` to synchronize the ghost with actual storage. This is safe because `unresolved external in _._ => NONDET` prevents storage havoc from low-level

calls, ensuring the ghost and storage remain in sync.
- **Sum-of-balances bounding**: In `Token.spec` preserved blocks, `require balanceOf(x) <= g_sumBalances` is a mathematical tautology — individual values cannot exceed their sum.

Assumptions - Proved

These invariants have been formally verified and are used as trusted preconditions in other rules via `requireInvariant`:

- `totalSupplyIsSumOfBalances` (T-1) — Used as a precondition in `transferConservation` (T-7) to ensure the sum-of-balances ghost is consistent with `totalSupply` before verifying transfer deltas. Without this, the prover could start from an arbitrary state where balances are inconsistent with the ghost sum.
- `totalSupplyBoundedByMaxSupply` (T-2) — Used in the `preserved mint(...)` block of `totalSupplyIsSumOfBalances` to bound `totalSupply` during the inductive step. This prevents the prover from considering states where `totalSupply` exceeds `maxSupply`, which would make the mint overflow check unreachable.

Assumptions - Unsafe

These scope reductions are necessary for prover tractability but may miss valid scenarios:

- **Bounded releaser array** (`loop_iter: 3`): Rules involving `addReleaser`, `removeReleaser`, and `payReleaseFee` constrain `releasers.length <= 3` because these functions use loops that the prover bounds to 3 iterations. Properties hold for arrays of length 0-3 but are not verified for larger arrays. Marked as `"UNSAFE: bounded by loop_iter"` in specs.
- **External call nondeterminism** (`NONDET`): Token methods (`transfer`, `transferFrom`, `balanceOf`, `mint`) and vault `release` on `PublicBridge` are summarized as `NONDET`. This means the prover does not verify the correctness of token/vault operations, only that the bridge logic handles their results correctly.

Verification Results

Final prover run URLs (Certora Prover v8.8.1). All 119 active properties verified; the 2 expected violations correspond to documented bugs (L-3).

Spec	Result	Prover URL
`Token`	All 17 rules verified	Prover Link
`PrivateChainBridge`	52 verified, 1 expected violation (B-F1/L-3)	Prover Link
`PublicBridge`	51 verified, 1 expected violation (P-F1/L-3)	Prover Link

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source); both modes share the same initial setup steps.

Common Setup (Steps 1-4)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup tutorial.

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx — installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match a specific prover version, pin it explicitly (e.g. `certora-cli==8.8.1`)

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.18
solc-select use 0.8.18
```

Remote Execution

5. Set up Certora key. You can get a free key through the Certora Discord or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
source ~/.bashrc
```

Note: If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:

```
certoraRun certora/conf/Token.conf --server production
```

Local Execution

Follow the full build instructions in the CertoraProver repository (v8.8.1). Once the local prover is installed it takes priority over the remote cloud by default. Tested on Ubuntu 24.04.

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
#   z3:   https://github.com/Z3Prover/z3/releases
#   cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.8.1
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Verification

Every `.conf` file includes `"wait_for_results": "all"` so the CLI waits and prints full results to the terminal.

Verify all contracts:

```
certoraRun certora/conf/Token.conf
certoraRun certora/conf/PrivateChainBridge.conf
certoraRun certora/conf/PublicBridge.conf
```

Run specific rules only:

```
certoraRun certora/conf/Token.conf --rule totalSupplyIsSumOfBalances onlyIssuerCanMint
certoraRun certora/conf/PrivateChainBridge.conf --rule nonceOnlyIncreases addReleaserIntegrity
certoraRun certora/conf/PublicBridge.conf --rule lockTokensRequiresPrivateDestination
```

Compilation check only (no cloud submission):

```
certoraRun certora/conf/Token.conf --compilation_steps_only
certoraRun certora/conf/PrivateChainBridge.conf --compilation_steps_only
certoraRun certora/conf/PublicBridge.conf --compilation_steps_only
```

Resources

To learn more about Certora formal verification:

- Updraft Assembly & Formal Verification Course — Comprehensive video course covering assembly and formal verification from the ground up
- Find Highs Using Certora Formal Verification — Practical guide with a companion repo containing simplified examples based on real code and bugs from private audits
- RareSkills Certora Book — Structured tutorial covering CVL syntax, patterns, and common pitfalls
- Alex FV Resources — Curated collection of formal verification resources, examples, and references
- Certora Tutorials — Official Certora documentation and guided tutorials